DOCUMENTATION DE RÉFÉRENCE

Le Workflow Core

Un cadre de collaboration entre intelligences artificielles spécialisées, piloté par l'humain, pour concevoir et faire évoluer des architectures de solution avec rigueur, traçabilité et sécurité.

Coordination multi-agents (A2A) | Validation humaine à chaque étape | Sécurité systématique | Mémoire & apprentissage | Portable & agnostique

1 Contexte & objectifs

Le **workflow Core** est la méthode de travail d'une équipe d'**agents d'intelligence artificielle spécialisés** en architecture de solution. Chaque demande — concevoir un système, faire évoluer une plateforme, documenter une architecture — est traitée comme un projet encadré, où un **coordinateur** orchestre des experts et où **l'humain décide à chaque jalon**.

Le contexte

Les projets d'architecture mêlent des expertises variées (solution, cloud AWS et Azure, données, sécurité) et exigent des décisions tracées et vérifiables. Le workflow Core apporte un cadre commun : une suite d'étapes claires, des rôles bien définis et une gouvernance qui garde l'humain aux commandes.

Les objectifs

- **Adapter l'effort au besoin** — léger pour le simple, complet pour le complexe.
- **Garantir la traçabilité** — chaque décision est consignée et justifiée, chaque besoin d'affaires est suivi jusqu'au livrable.
- **Sécuriser par défaut** — un contrôle sécurité à chaque modification.
- **Garder l'humain décideur** — rien n'avance sans sa validation.
- **Apprendre dans le temps** — les corrections deviennent des règles durables.

Principe fondateur : le workflow s'adapte au travail, et non l'inverse. Une modification simple reçoit un traitement minimal ; un chantier complexe ou à risque reçoit un traitement complet. L'effort est calibré selon l'intention, l'existant, la complexité et le risque.

2 Fonctionnement global : cinq phases

Un projet avance en **cinq phases successives**. Entre chaque phase, un **point de contrôle** (le « gate ») laisse l'humain approuver avant de continuer. Plus on avance, plus la validation est exigeante.

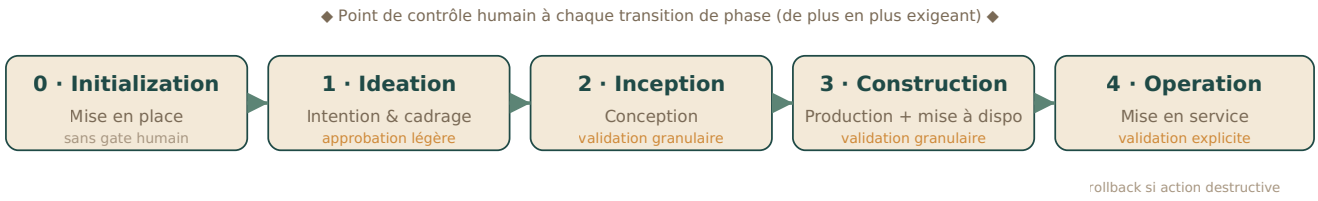


Figure 1 — Les cinq phases du workflow et l'exigence croissante de validation humaine.

Phase	Ce qui s'y passe	Contrôle humain
0 · Initialization	Vérifications de départ : répertoire du projet, contexte Git (branche dédiée à l'issue si projet Git), projet neuf ou existant, ouverture de la piste d'audit.	Automatique aucun gate
1 · Ideation	Capture de l'intention, contraintes et faisabilité, définition du périmètre, maquettes éventuelles.	Approbation de l'intention et du périmètre **léger**
2 · Inception	Cadrage, analyse des besoins (classés MoSCoW et identifiés BA-…), découpage en livrables, conception et décisions structurantes (ADR d'abord sur impact structurant).	Validation **choix par choix**
3 · Construction	Walking skeleton, production détaillée, contrôle cohérence + sécurité, consolidation, puis mise à disposition (commit + PR ou archivage).	Validation **choix par choix**

4 · Operation	Déploiement sous validation, notification de fin, maintenance et support.	Validation **explicite** + rollback

La phase **Construction** se décompose en cinq étapes : walking skeleton (première tranche de bout en bout), production détaillée des livrables, **contrôle cohérence + sécurité** (revue de cohérence puis revue de sécurité adversariale), consolidation, et enfin **mise à disposition** — par commit + Pull Request ou par archivage, suivie d'une notification de fin de réalisation.

3 Les méthodes du workflow

3.1 — Le cycle d'une étape (6 temps)

Chaque étape (ou « stage ») suit le même rythme. Ce canevas garantit qu'aucune production n'échappe au contrôle qualité, sécurité et humain.

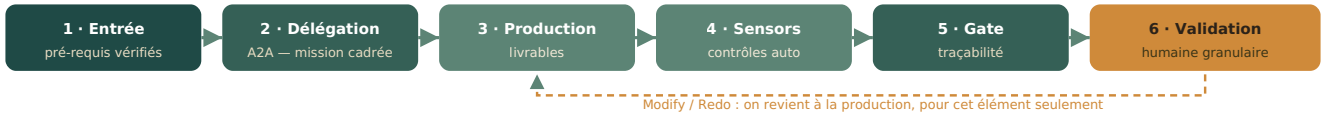


Figure 2 — Le cycle en 6 temps d'une étape, de l'entrée à la validation humaine.

3.2 — La décision humaine : Keep / Modify / Redo

À chaque point de validation, le coordinateur présente **chaque choix séparément** (le choix, sa justification, une alternative). L'humain tranche **élément par élément** :

Keep Le choix est validé, on avance.

Modify L'humain reformule, on réajuste.

Redo Le choix est rejeté, on propose une alternative.

Aucun élément non validé n'avance. Jamais d'approbation globale « tout ou rien ».

3.3 — Calibrer l'effort : scopes & axes

Le principe « le workflow s'adapte au travail » est outillé par des **scopes** (le type de chantier, qui détermine les étapes exécutées) et par **deux réglages indépendants** : la **profondeur** du détail produit et le **niveau de vérification**.

8 scopes types

standard (défaut) · **feature** · **infra** · **security-patch** · **mvp** · **poc** · **express** · **enterprise**

Le scope est auto-détecté puis **confirmé explicitement** avant de démarrer — jamais de démarrage silencieux.

2 axes de réglage

- **Profondeur** : minimal · standard · comprehensive
- **Vérification** : advisory · standard · renforcé

Un niveau lié à la sécurité ne peut **jamais** être abaissé sans validation humaine tracée.

Détail de chaque scope : annexe (section 10).

3.4 — La mémoire de règles (apprentissage)

Le workflow **apprend des corrections humaines** : quand une personne corrige ou rejette un choix, cela devient un **candidat-règle**. Après confirmation explicite, la règle est écrite et **s'appliquera au prochain projet** — jamais rétroactivement. La mémoire est organisée en **quatre couches** de priorité décroissante.

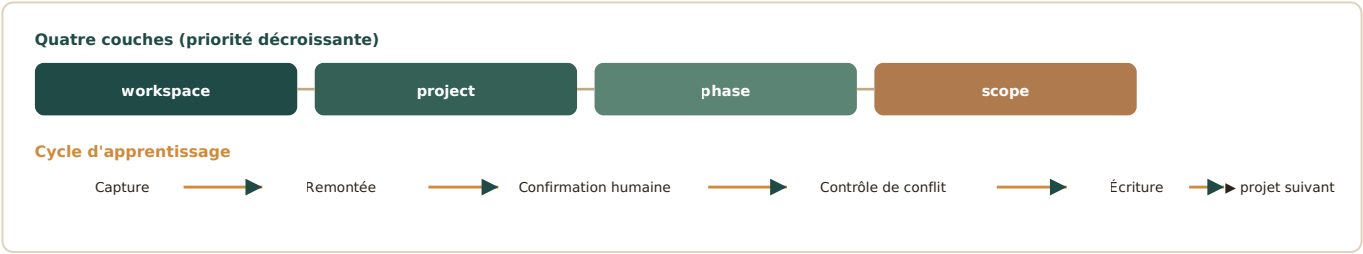


Figure 3 — Couches de la mémoire de règles et cycle d'apprentissage.

3.5 — Garde-fous & contrôles automatiques

Invariants non contournables

- Validation humaine granulaire
- Décision structurante tracée (registre de décisions)
- ADR obligatoire sur impact structurant (ADR d'abord)
- Piste d'audit tenue sur l'issue
- Contrôle sécurité systématique (OWASP / STRIDE)
- Aucune action à impact sans validation ; rollback avant action destructive
- Blocage ⇒ mention explicite de l'humain

Gates & sensors (advisory)

Des contrôles automatiques factualisent la **traçabilité** (sections obligatoires, cycle de vie des données, couverture de la demande, validité des diagrammes, sources des affirmations, chaîne besoin d'affaires (MoSCoW) → exigence → décision → livrable).

Ils **ne bloquent jamais** et **ne remplacent ni** la validation humaine **ni** le contrôle sécurité : ils assistent la décision, qui reste humaine.

3.6 — Le format de retour d'un agent

Tout retour d'un agent sur l'issue (compte rendu, retour A2A) adopte une **prose réduite** — phrases courtes, une idée par puce, aucun remplissage — et une structure en **trois sections** :

- **Résumé** — synthèse des tâches effectuées, en puces (toujours présent).
- **Points bloquants** — uniquement s'il en existe.
- **Action humaine requise** — uniquement si l'humain doit intervenir ; comporte alors la liste des fichiers modifiés et une mention humaine active.

La section « Action humaine requise » n'apparaît **jamais** sur un simple retour entre agents (A2A) : rendre la main à un agent n'est pas une action humaine.

La concision ne relâche aucune règle de gating ; elle met l'arbitrage en forme, sans s'y substituer.

3.7 — Le chargement paresseux (lazy loading)

Un agent dispose d'une fenêtre de lecture limitée. Plutôt que de tout charger d'emblée, le workflow applique le **chargement paresseux** : on ne lit au départ que de **légers inventaires** (titres, index, descriptions), et le contenu complet d'un document n'est ouvert que **lorsque l'étape qui en a besoin est déclenchée**. Cela préserve la fenêtre de contexte, accélère le démarrage et évite de polluer l'analyse avec des informations hors sujet.

Au démarrage (léger uniquement)

- liste des livrables du projet ;
- agents disponibles et leur **description** (pas leurs instructions) ;
- compétences et leurs descriptions ;
- index des titres du registre de décisions ;
- sommaire de la documentation d'architecture.

À la demande (au déclenchement)

- contenu complet d'un agent, d'une compétence, d'une méthodologie ou d'un gabarit ;
- corps d'un document de décision ou d'une spécification ;
- **fiche de stage et protocole** de l'étape en cours ;
- règles de la phase et du scope confirmé.

Le rôle des front matter

Ce qui rend la lecture légère possible, c'est le **front matter** : un court bloc d'en-tête (au format YAML, encadré par deux lignes `---`) placé en tête de chaque fichier du workflow. Il **décrit le fichier sans en livrer le contenu** — son nom, sa description, ses mots-clés, ses liens. Le workflow lit d'abord ces en-têtes pour savoir **quoi** ouvrir, puis n'ouvre le **corps** que lorsque l'étape le demande.

Fichier	Ce que son front matter expose	Ce qui est lu d'abord, puis à la demande
Agent	nom, libellé, **description**, compétences assignées	Seule la **description** est lue au démarrage (sous Multica, le champ « description » de l'agent) ; ses **instructions** détaillées ne sont pas chargées par le coordinateur.
Compétence (skill)	nom, **description** (quand l'utiliser)	Le nom et la description servent d'inventaire ; le contenu de la compétence et ses références ne sont ouverts que par l'agent qui l'applique.
Workflow — fiche de stage	phase, condition d'exécution, agents, gate humain, scopes, sensors, entrées / sorties	L'en-tête sert à **enchaîner les étapes** ; le corps (déroulé de l'étape) n'est lu que par l'agent qui exécute l'étape, sur indication de la délégation.
Workflow — scope	nom, Depth, Vérification, **mots-clés**, description	Les mots-clés alimentent l'**auto-détection** du scope ; ses règles propres ne sont chargées qu'une fois le scope confirmé.
Workflow — sensor	identifiant, commande, moment de déclenchement, fichiers visés	L'en-tête dit **quand et sur quoi** lancer le contrôle ; le contrat de vérification n'est exécuté qu'à ce moment-là.
Documentation d'architecture	thème, domaine, **sujets** (titres de sections), types de présentation, sensibilité	Pour **lire, modifier ou créer**, on lit d'abord les en-têtes pour cibler le fichier et la section ; seuls ceux-là sont ouverts. Si rien ne couvre le besoin, on part du gabarit au lieu de dupliquer un thème.
Décision (ADR)	auteurs, acceptation, remplacements (*supersedes*)	Le registre est parcouru par ses titres ; l'en-tête est vérifié par les contrôles automatiques ; le corps de la décision n'est lu que lorsqu'elle est utile.

Un front matter tenu à jour est donc une condition du chargement paresseux : la liste des `sujets` d'un document d'architecture est mise à jour dans le même changement que tout ajout, renommage ou suppression de section, et le contrôle automatique **sujets-a-jour** signale à l'écriture tout écart (advisory). La règle de ciblage est portée par la compétence du gabarit, pas par le workflow : un autre gabarit peut définir ses propres champs.

Règle complémentaire	Comment elle s'applique
Fiches de stage & protocoles	Chaque délégation indique au délégataire la ou les fiches à charger pour sa mission : il ne lit jamais l'ensemble.
Règles de phase et de scope	Ouvertes uniquement quand la phase est déclenchée ou le scope confirmé ; une règle non applicable est marquée **N/A** et n'est pas chargée.

Normes de sécurité spécifiques	PCI DSS, GDPR, Loi 25, LPRPDE : chargées seulement si explicitement demandées ; par défaut, OWASP et STRIDE.

> **Traçabilité.** Ce qui a été chargé est consigné sur l'issue (piste d'audit) : on sait à tout moment sur quelles informations repose une décision. Le chargement paresseux ne supprime aucun contrôle — il décide seulement *quand* lire, jamais *s'il faut* respecter une règle.

4 Les agents & les compétences mobilisés

Le workflow réunit une **équipe d'agents spécialisés**, coordonnée par un chef d'orchestre. Le **coordinateur ne produit pas les livrables** : il analyse, découpe, délègue, contrôle et fait valider. Les experts produisent dans leur domaine.

Agent (fonction)	À quoi il sert
Architecture Solution & Intégration coordinateur	Orchestre le workflow : analyse la demande, délègue, sollicite les contrôles, demande les validations humaines, met les livrables à disposition.
Architecte de solution	Conçoit les solutions : documentation d'architecture, décisions structurantes, diagrammes.
Architecte AWS	Définit les services cloud AWS requis, produit les diagrammes et optimise les coûts.
Architecte Azure	Porte toutes les fonctions cloud Microsoft Azure : services, diagrammes, coûts (CAPEX / OPEX), et livrables d'infrastructure de déploiement Azure (IaC Bicep / ARM / Terraform, plans de déploiement et de rollback).
Architecte Cybersécurité	Conçoit et renforce la sécurité (OWASP, STRIDE, normes sur demande).
Architecte de données	Modélise les données, le cycle de vie, la gouvernance et la classification.
OpenSpec Expert	Pilote la méthode spec-driven lorsqu'elle est activée.
Reviewer de cohérence	Vérifie la cohérence documentation ↔ décisions et la longueur de prose des livrables (revue seule, aucun livrable).
Reviewer de sécurité	Rend le verdict de revue sécurité (adversariale) — obligatoire dès qu'une surface de sécurité évolue.
Infrastructure Windows	Administration des postes et de l'infrastructure Windows (migrations, Intune, VM, golden image, Autopilot, SCCM). Périmètre strictement Windows ; le cloud Azure relève de l'Architecte Azure.
Vente & Appels d'Offres	Synthétise les architectures en supports de vente et réponses d'appels d'offres.
Présentation client	Produit des présentations ciblées à partir des documents validés.
Experte d'archivage	Import / export de documents, mise à disposition et archivage des livrables validés, et mise à disposition par Git (commit + Pull Request) sur demande.
Agent de notifications	Notifie la fin de réalisation et de déploiement, sur demande du coordinateur.

Quatorze agents au total. La correspondance nom ↔ fonction fait autorité dans la compétence **project-defaults** (source unique) ; chaque agent est porté par sa définition propre.

Les compétences (« skills ») partagées

- **project-defaults** — structure de projet, conventions de nommage et correspondance des agents (source unique).
- **architecture-solution-gabarits** — modèles standardisés des documents d'architecture.
- **architecture-securite-gabarits** — gabarits de documentation sécurité (00 → 06) et revue de sécurité SG-3.
- **create-architecture-decision-record** — format normalisé des décisions structurantes (ADR).
- **aws-solution-architect** — patrons et gabarits d'architecture AWS.
- **azure-solution-architect** — patrons serverless et gabarits IaC Azure (Bicep / ARM / Terraform azurerm).
- **git-delivery** — mise à disposition par Git : commit des fichiers et ouverture d'une Pull Request vers la branche cible.
- **openspec-proposal-creation** — propositions OpenSpec et matrice de traçabilité MoSCoW (section 9).
- **cdae-ai-eligibilite** — analyse d'éligibilité au crédit d'impôt CDAE-IA (sur demande).

Une compétence est une référence commune que les agents consultent pour travailler de façon cohérente d'un projet à l'autre.

5 L'A2A : comment les agents collaborent

« A2A » désigne la collaboration **d'agent à agent**. Dans le workflow, elle repose sur un principe simple : **tout passe par l'issue** (le fil du projet). Le coordinateur délègue en publiant un commentaire qui **mentionne** l'agent voulu et lui confie une **mission cadrée** (objectif, périmètre, critères d'acceptation). L'agent produit, puis **rend la main par une mention active** vers l'assigneur pour contrôle — c'est l'agent délégataire qui ferme la boucle A2A.

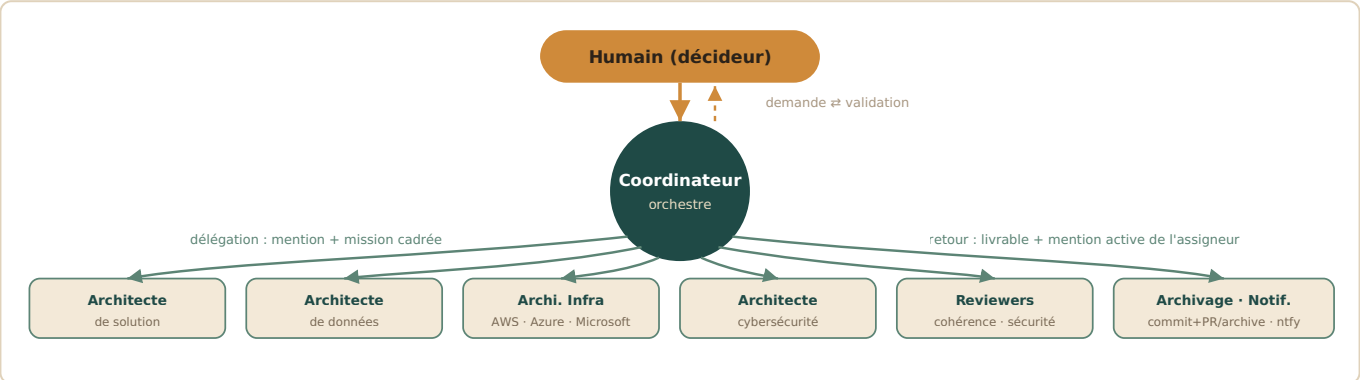


Figure 4 — Le coordinateur délègue en « étoile » (hub-and-spoke) ; chaque livraison revient pour contrôle avant validation humaine.

5.1 — États & interactions : A2A, Agent ↔ Humain, gates

Au-delà de la topologie « en étoile », le travail est une **séquence d'interactions** rythmée par les **gates** et par l'**état de l'issue**. Le diagramme se lit de haut en bas : chaque **flèche horizontale** relie deux acteurs — en **vert** l'**A2A** (entre agents), en **ocre** l'**Agent ↔ Humain** — chaque **losange** est un gate, et la colonne de droite donne l'**état de l'issue**. Les acteurs qui **construisent** et celui qui **vérifie** ont chacun leur couloir, et la **mise à disposition** puis la **notification** finale sont explicites.

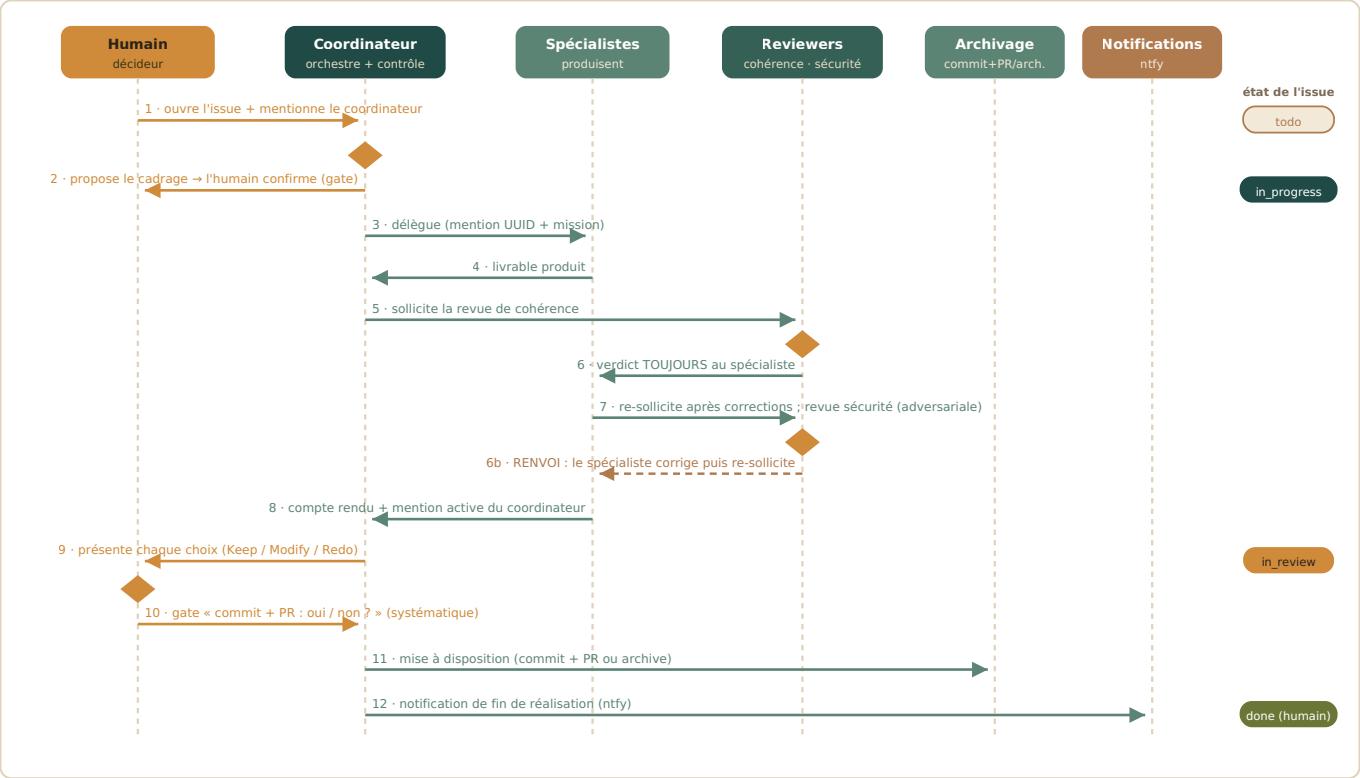


Figure 4bis — États & interactions : couloirs Humain · coordinateur · spécialistes · reviewers · archivage · notifications ; revues séquentielles (cohérence puis sécurité) dont le verdict revient toujours au spécialiste ; gate « commit + PR » systématique avant mise à disposition ; colonne de droite = état de l'issue, boucles RENVOI / Modify-Redo.

Les règles de la collaboration

- Déclenchement par **mention valide** sur l'issue + mission claire.
- L'agent délégataire **rend la main par une mention active** de l'assigneur, et ferme ainsi la boucle A2A.
- Les revues (cohérence puis sécurité) sont **séquentielles** ; leur verdict revient toujours au spécialiste, jamais à un autre reviewer ni au coordinateur.
- Les reviewers analysent la longueur de prose du livrable et demandent sa réduction localisée.
- Un contrôle de sécurité est sollicité à **chaque modification** d'architecture (plancher SG-3, non substituable).

Quatre façons de collaborer

- **En étoile** — chaque expert travaille isolément pour le coordinateur.
- **En chaîne** — les experts se passent le relais dans l'ordre.
- **En nuée** — plusieurs experts challengent un même brouillon.
- **En direct** — le coordinateur exécute lui-même l'étape.

Le flux ADR, d'abord. Toute demande à impact structurant déclenche d'abord un ADR (issue parente dédiée) : production → revues cohérence + sécurité → ADR Proposée → décision humaine. Accepté, l'ADR lance ses sous-issues de livrable ; une fois tous les livrables validés et mis à disposition, l'ADR passe à « ADR Acceptée » de façon dérivée. Refusé, l'ADR et l'issue d'origine sont annulés en cascade (section 8).

Toujours sous contrôle humain. Quelle que soit la façon de collaborer, dès qu'un blocage survient, l'agent concerné **mentionne explicitement l'humain** et met le projet en pause : il n'avance jamais en devinant. La décision finale appartient toujours à la personne.

6 Utiliser le workflow avec Multica

Sur Multica, le workflow s'exécute **autour de l'issue** : chaque demande ouvre une issue, et **tout le travail — analyse, délégation, décisions, validations — vit en commentaires** sur cette issue, qui tient lieu de piste d'audit. L'humain déclenche, suit et valide depuis ce même fil.

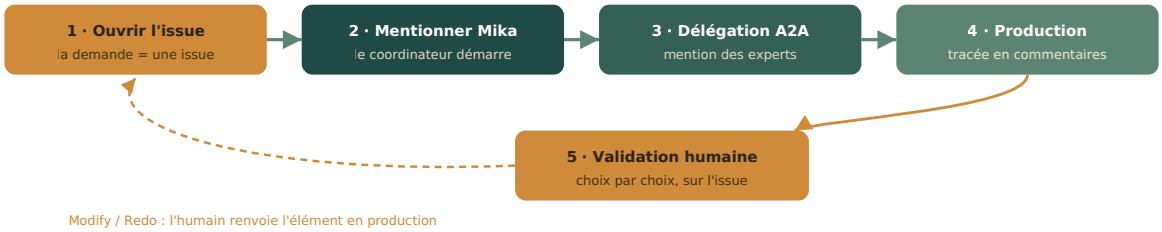


Figure 5 — Le parcours d'une demande sur Multica, de l'ouverture de l'issue à la validation humaine.

Les mécanismes Multica mobilisés

- **L'issue comme pivot** — statut, priorité, assignation et historique au même endroit ; statuts dédiés « ADR Proposée » et « ADR Acceptée » pour le flux de décision.
- **La mention A2A** — déléguer, c'est mentionner l'agent voulu avec une mission cadrée ; l'identité de l'agent est résolue automatiquement, jamais devinée.
- **La piste d'audit** — chaque étape est un commentaire ; l'historique n'est jamais écrasé.
- **Les pièces jointes** — les livrables (documents, diagrammes, PDF) sont attachés à l'issue ; un commit + Pull Request est aussi proposé systématiquement à la mise à disposition.

Mettre le workflow en service

- **Agents** — créer l'équipe (coordinateur + spécialistes), chacun porté par sa définition.
- **Compétences** — les skills sont d'abord **importées** dans l'espace de travail, puis **assignées** aux agents : une compétence présente dans le dépôt n'est pas active tant qu'elle n'est pas importée.
- **Démarrage** — ouvrir l'issue, décrire le besoin, mentionner le coordinateur : le workflow se déroule et sollicite l'humain à chaque jalon.

L'humain reste aux commandes depuis l'issue. Rien n'avance sans sa validation, aucune action à impact n'est engagée sans accord explicite — y compris le commit + Pull Request, demandé par un gate distinct — et tout blocage le mentionne directement pour qu'il tranche.

7 Un workflow agnostique : au-delà de Multica

Le workflow n'est **pas lié à un moteur d'exécution particulier**. Il est entièrement décrit en **documents Markdown lisibles** — les instructions du coordinateur, les fiches d'étape, les protocoles, la mémoire de règles, les scopes et les contrôles. Cette **forme déclarative** est sa force : **la méthode est le contenu, pas l'outil** qui l'exécute.

Deux niveaux d'agnosticité

- **Agnostique de méthodologie** — aucune méthode imposée ; une approche spec-driven (p. ex. OpenSpec) s'active seulement si le projet la déclare.
- **Agnostique d'outillage** — la description est volontairement séparée de tout moteur technique propriétaire : on adopte **la forme, pas la machinerie**.

Ce qu'un harnais doit fournir

- Lire des **instructions en Markdown** et les suivre.
- Faire tourner des **agents spécialisés** et les **faire dialoguer** (A2A).
- Un **fil partagé** pour tracer les décisions et solliciter l'humain.

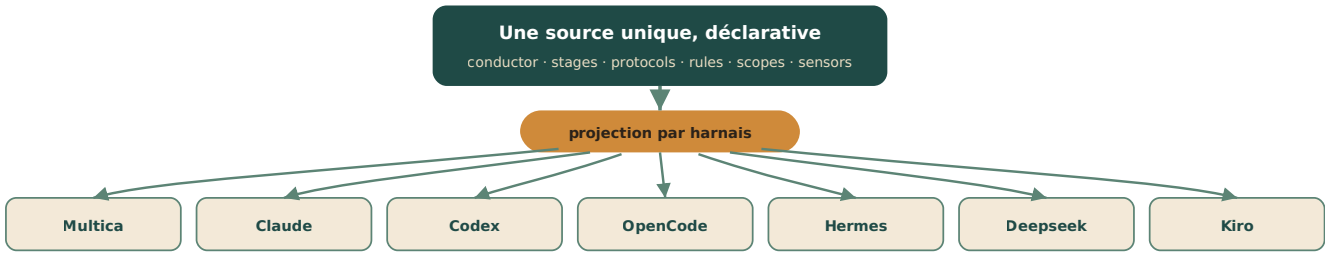


Figure 6 — La même source alimente différents harnais ; seule la couche d'adaptation change, jamais la méthode.

Comment l'utiliser avec un autre harnais

Le passage d'un harnais à l'autre ne touche **ni les phases, ni les étapes, ni les garde-fous** : seule change la **fine couche d'adaptation** qui relie la méthode aux spécificités de l'outil (format d'instruction d'agent, mécanisme de messages, façon de solliciter l'humain).

Harnais	Ce qui change (couche d'adaptation)	Ce qui ne change jamais
Multica	Issue + commentaires, mentions A2A, pièces jointes — l'exécution native décrite en section 6.	Les 5 phases, le cycle d'étape, la décision Keep / Modify / Redo, les scopes & axes, la mémoire de règles, le contrôle sécurité et tous les invariants.
Claude	Fichiers d'instructions et de sous-agents au format attendu par l'outil ; options spécifiques projetées depuis la source.	
Codex	Instructions projetées dans sa configuration d'agent ; messages d'étape adaptés à sa surface.	
OpenCode	Définitions d'agents et déclenchements adaptés à son modèle d'exécution.	
Hermes	Même source, projetée dans son mécanisme d'orchestration et de messages.	
Deepseek Harness	Instructions et définitions d'agents projetées dans son format ; messages d'étape adaptés à sa surface d'exécution.	
Kiro	Agents et instructions portés dans sa configuration native ; déclenchements et sollicitations humaines adaptés à son moteur.	

Une méthode, plusieurs moteurs. Parce que la rigueur vit dans les documents — et non dans un outil — le workflow garde **le même comportement, les mêmes garde-fous et la même qualité** quel que soit le harnais qui l'exécute. On change de moteur sans réécrire la méthode.

8 Le flux ADR : décider avant de construire

Une **décision structurante** — un choix d'architecture dont l'impact dépasse le livrable du moment (une technologie de base de données, un découpage de services, un modèle de sécurité) — n'est jamais prise à la volée. Elle est tracée dans un **ADR** (Architecture Decision Record) : un document court qui consigne le choix, sa justification et les alternatives écartées. Le workflow en fait un invariant : **dès qu'une demande a un impact structurant, l'ADR passe en premier — avant tout livrable.**

> **« ADR d'abord ».** Si, au cadrage, l'impact d'une demande est jugé **structurant**, le workflow ouvre une **issue ADR dédiée** (issue parente de la décision) **avant** de produire quoi que ce soit. En cas de doute, on traite comme structurant : c'est un plancher, jamais un plafond.

8.1 — Les six temps du flux

L'ADR suit un cycle court et toujours identique, où la décision finale revient à l'humain et où la **mise à disposition précède la clôture** :

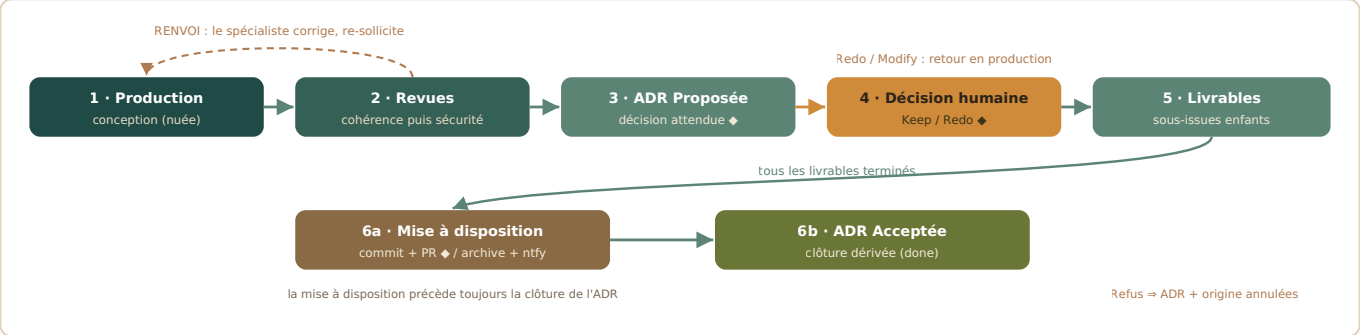


Figure 7 — Le flux ADR : production → revues → ADR Proposée → décision humaine → livrables → mise à disposition puis clôture « ADR Acceptée ». Un refus annule l'ADR et l'issue d'origine en cascade.

1. **Production** — l'Architecte de solution rédige la conception et la décision ; les autres experts la challengent (mode « nuée »). La décision est tracée dans le registre du projet.
2. **Revues — cohérence puis sécurité**, séquentielles ; le verdict revient toujours au spécialiste auteur, qui corrige et re-sollicite sur RENVOI. L'auteur n'est jamais son propre relecteur.
3. **ADR Proposée** — une fois les deux revues indépendantes intégrées, l'ADR est posée en proposition : une **décision humaine est attendue**.
4. **Décision humaine** — au gate granulaire, l'humain **accepte** (Keep) ou **rejette** (Redo). Rien n'est écrit « Accepté » sans cet arbitrage.
5. **Livrables** — une fois l'ADR acceptée, ses sous-issues de livrable (enfants) sont créées et lancées ; l'issue ADR reste **bloquée** tant qu'elles tournent.
6. **Mise à disposition puis clôture** — quand tous les livrables sont terminés, ils sont **d'abord** mis à disposition (gate « commit + PR : oui / non ? », ou archivage, puis notification), **ensuite** l'ADR passe à « ADR Acceptée » de façon dérivée.

8.2 — Le cycle de vie reflété sur l'issue (Multica)

Sur Multica, le **statut de l'issue** qui porte l'ADR suit en temps réel la phase du flux. L'agent qui fait avancer l'ADR met à jour ce statut dans le même temps, pour que le tableau montre l'ADR là où elle en est.

Phase du flux	Statut de l'issue	Ce que cela signale
Production de la conception	`in_progress`	L'ADR est en cours de rédaction / correction.
Revues (cohérence, sécurité)	`in_review`	L'ADR est remise aux reviewers indépendants.
Proposée à l'humain	`ADR Proposée`	Les revues sont intégrées ; une décision humaine est attendue.

Acceptée (Keep)	**blocked** (parente)	Les sous-issues de livrable sont lancées ; l'ADR reste ouverte jusqu'à leur fin.
Rejetée (Redo)	**cancelled** + origine en cascade	Aucun livrable ; l'issue d'origine est annulée elle aussi.
Tous les livrables terminés	**ADR Acceptée** → origine **done**	Clôture dérivée, après mise à disposition.

Garde-fous du flux. Le passage en « ADR Proposée » exige **deux revues indépendantes** (cohérence et sécurité), postées par des reviewers qui ne sont pas l'auteur. L'acceptation reste un **acte humain** — jamais automatique — et la mise à disposition précède toujours la clôture « ADR Acceptée ». Un refus annule l'ADR et l'issue d'origine en cascade : un impact structurant n'est jamais construit sans décision assumée.

9 Traçabilité des exigences d'affaires & MoSCoW

Une architecture n'a de valeur que si l'on peut montrer **pourquoi** chaque élément existe. Le workflow relie donc chaque **besoin d'affaires** à ses **exigences**, à ses **scénarios**, puis aux **décisions (ADR)** et **livrables** qui y répondent, et classe chaque besoin par **priorité de livraison** selon la méthode **MoSCoW**. Résultat : on peut toujours remonter d'un livrable à un besoin, ou descendre d'un besoin à ce qui le couvre.

Deux repères stables

- **Identifiant `BA-…`** — chaque besoin d'affaires retenu en porte un, stable, repris tel quel dans les exigences, les specs, les ADR et les livrables.
- **Priorité MoSCoW** — **Must** **Should** **Could** **Won't** : ce qui est indispensable, important, souhaitable ou exclu de cette livraison.

À ne pas confondre

La priorité **MoSCoW** dit quand et si on livre (priorité de livraison). Les mots **SHALL / SHOULD / MAY** du format EARS disent à quel point la phrase d'une exigence est contraignante. Les deux coexistent sur une même exigence sans se substituer.

9.1 — La chaîne de traçabilité

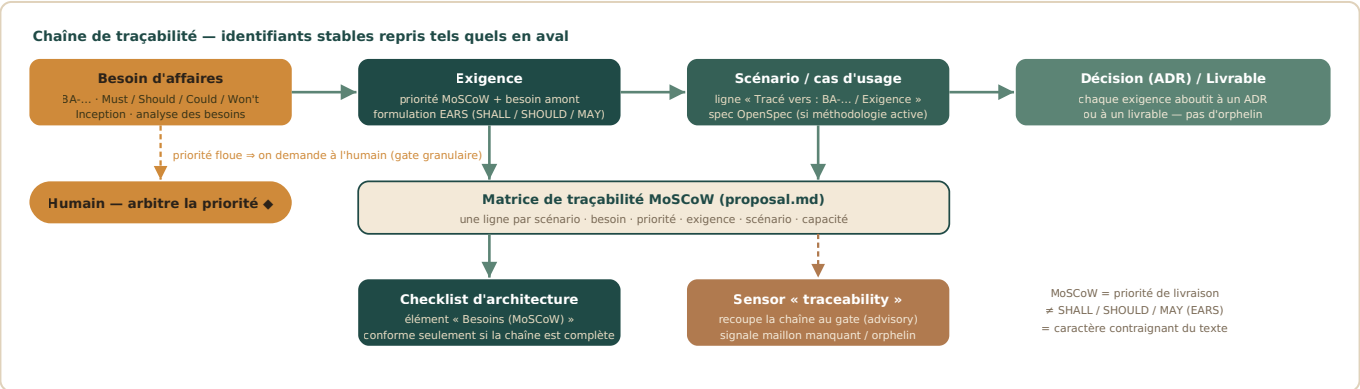


Figure 8 — Du besoin d'affaires (BA-…, priorisé MoSCoW) jusqu'au livrable ; la matrice MoSCoW est l'élément récapitulatif qui alimente la checklist d'architecture et que le sensor traceability recoupe.

9.2 — Qui fait quoi, à quelle étape

Étape / outil	Rôle dans la traçabilité	Contrôle humain
Inception — analyse des besoins	Chaque besoin d'affaires retenu reçoit un identifiant `BA-…` et une priorité MoSCoW. C'est la **source amont** de toute la chaîne.	Priorité floue ⇒ l'humain tranche au gate granulaire

Proposition OpenSpec si la méthodologie est active	Chaque exigence porte sa priorité MoSCoW, son besoin amont `BA-…` et, si besoin, son exigence amont ; chaque scénario porte une ligne « Tracé vers ». Le tout est consolidé dans la **Matrice de traçabilité MoSCoW** du `proposal.md` (une ligne par scénario : besoin · priorité · exigence · scénario · capacité).	Matrice présentée à l'approbation de la proposition
Checklist d'architecture	Le coordinateur lit la matrice comme **élément récapitulatif** pour suivre la conformité de la section « Besoins (MoSCoW) » (entreprises et affaires · solutions · technologique · sécurité).	Avancement en % suivi à chaque gate
Sensor traceability	Recoupe la chaîne complète au gate : besoin priorisé et relié à une exigence, exigence reliée à un ADR ou un livrable, scénario relié à son besoin, aucune décision ni livrable orphelin.	Advisory : signale, ne bloque pas

9.3 — Les règles à retenir

- **Pas d'orphelin.** Tout scénario renvoie à une exigence ; toute exigence renvoie à un besoin d'affaires et porte une priorité MoSCoW.
- **Ne jamais deviner.** Si la priorité d'un besoin n'est pas claire, elle est demandée à l'humain, jamais inventée.
- **Identifiants stables.** Les `BA-…` et les identifiants d'exigence ne changent pas, pour que la chaîne reste résolvable.

- **Chaîne incomplète = non conforme.** Un besoin sans priorité, ou un scénario sans renvoi à son besoin, ne compte pas comme conforme dans la checklist.
- **Aide à la décision, pas substitut.** Le sensor est advisory et ne remplace ni la validation humaine ni le contrôle sécurité.
- **Sans OpenSpec**, la priorisation MoSCoW et les `BA-…` sont tout de même tracés au stage d'analyse des besoins.

10 Annexe — Les scopes en détail

Un **scope** est le **type de chantier** : il fixe le parcours d'étapes et porte des **valeurs par défaut** d'axes — **Depth** (combien on écrit) et **Vérification** (à quel point on vérifie). Cette annexe détaille chaque scope du workflow avec un **diagramme d'état** de son **flux spécifique**, **interactions A2A** (**vert**) et **Humain** (**ocre**) mises en avant.

Le scope est auto-détecté par mots-clés puis **confirmé explicitement**. En cas de correspondances multiples, l'ordre de priorité est : `security-patch` > `enterprise` > `infra` > `feature` > `mvp` > `poc` > `express` > `standard`. Quels que soient le scope et les axes, les **invariants restent actifs** : validation humaine granulaire, décision structurante tracée, piste d'audit, et plancher OWASP / STRIDE jamais désactivable.

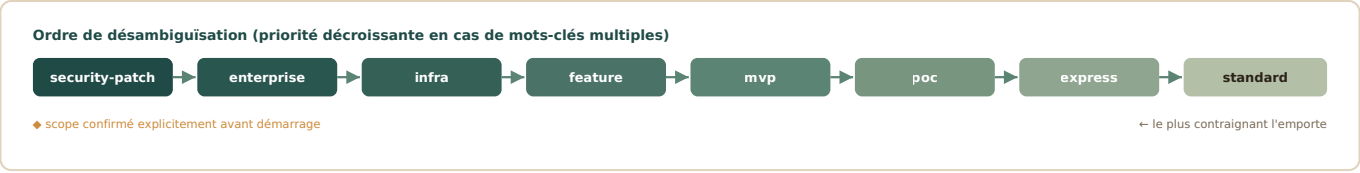


Figure 9 — Ordre de priorité de désambiguïsation des scopes (le plus contraignant l'emporte).

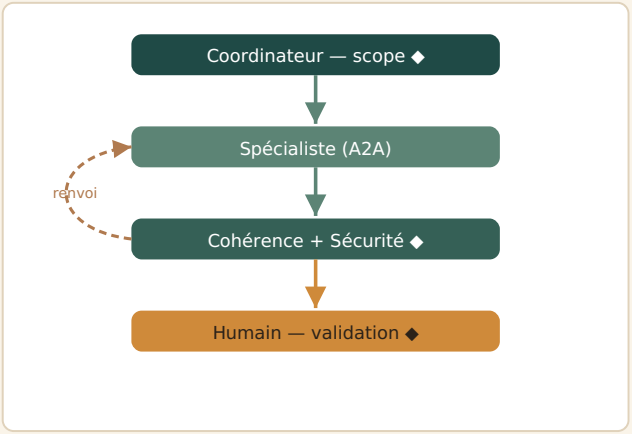
Légende des diagrammes d'état par scope : ▬ interaction A2A (agent → agent) · ▬ interaction Agent ↔ Humain · ▬ ▬ boucle de retour (renvoi / Modify / escalade) · ◆ gate / validation.

`standard` défaut

Mots-clés : architecture, conception, évolution, refonte, design — **Depth standard** **Vérif. standard**

Parcours d'architecture **complet** pour une conception ou une évolution « normale » — scope par défaut en l'absence de mot-clé. Tous les stages d'Ideation, Inception et Construction s'exécutent ; l'Operation est conditionnelle (seulement s'il y a une action à impact).

Flux A2A & Humain. Le coordinateur délègue aux spécialistes, sollicite les revues (cohérence puis sécurité), puis fait valider l'humain choix par choix. Aucun court-circuit.



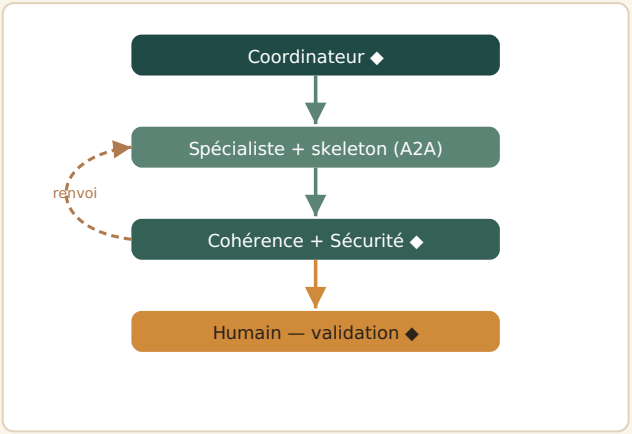
Parcours complet : A2A → revues → validation humaine

`feature` ajout fonctionnel

Mots-clés : fonctionnalité, feature, ajout, évolution fonctionnelle — **Depth standard** **Vérif. standard**

Ajout ou évolution fonctionnelle : **Inception ciblée** puis Construction, avec **contrôle sécurité systématique**. Le chargement du contexte existant est activé (on part d'un existant) et le **walking skeleton** est pertinent quand la Construction ouvre sur un squelette fonctionnel.

Flux A2A & Humain. Délégation aux spécialistes, revues cohérence + sécurité, validation humaine granulaire — comme le parcours standard, resserré sur la fonction visée.



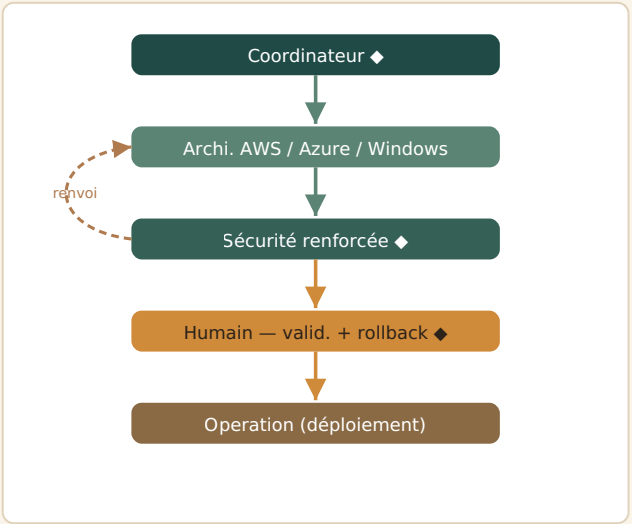
Inception ciblée + Construction, sécurité systématique

`infra` plateforme

Mots-clés : infrastructure, réseau, AWS, Azure, migration, Windows, plateforme, déploiement — **Depth standard** **Vérif. renforcé**

Travaux d'infrastructure ou de plateforme. Accent sur l'**Architecte AWS**, l'**Architecte Azure** et l'**Infrastructure Windows** ; **rollback obligatoire** et **Operation activée** (action à impact). Vérification **renforcée** du contrôle des livrables.

Flux A2A & Humain. Le coordinateur délègue aux architectes infra, exige un plan de déploiement et de rollback, fait contrôler la sécurité, puis l'humain valide avant toute action à impact.



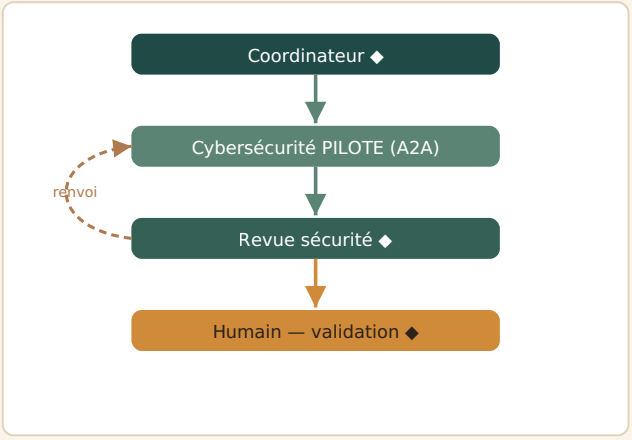
Architectes infra, rollback + Operation, sécurité renforcée

`security-patch` priorité max

Mots-clés : sécurité, correctif, patch, vulnérabilité, CVE, durcissement, hardening, faille — **Depth standard** **Vérif. renforcé**

Correction ou durcissement de sécurité : l'**Architecte cybersécurité pilote**, périmètre resserré, traçabilité renforcée, analyse d'impact / non-régression produite avant toute recommandation. **Garde-fou non abaissable** : Depth ≥ standard, Vérification ≥ renforcé ; aucun plafond de revue ne s'applique.

Flux A2A & Humain. La cybersécurité mène, la revue de sécurité reste obligatoire, puis validation humaine. Aucun override ne peut abaisser le niveau de contrôle.



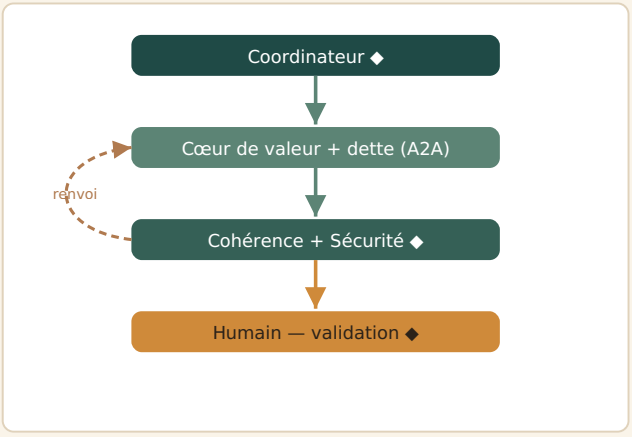
Cybersécurité pilote, contrôle non abaissable

`mvp` cœur de valeur

Mots-clés : MVP, produit minimum viable, cœur de valeur, first cut — **Depth standard** **Vérif. standard**

Produit minimum viable : on livre le **cœur de valeur** à Depth standard, en **traçant la dette technique** contractée. Le contrôle sécurité est maintenu ; le walking skeleton est pertinent.

Flux A2A & Humain. Délégation, revues, validation humaine — comme standard, avec un focus explicite sur le cœur de valeur et une trace de la dette reportée.



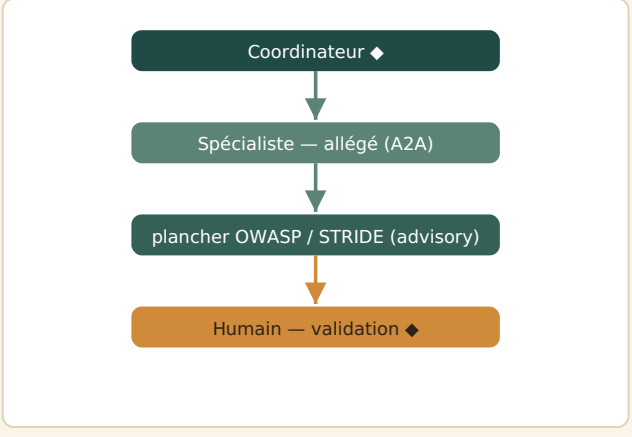
Cœur de valeur, dette tracée, sécurité maintenue

`poc` jetable

Mots-clés : PoC, preuve de concept, prototype, jetable, spike, expérimentation — **Depth minimal** **Vérif. advisory**

Preuve de concept **jetable** : parcours allégé, Depth minimale, vérification advisory. Un PoC est **non promouvable tel quel** — toute reprise en feature / mvp / enterprise re-déclenche le contrôle sécurité complet du scope cible. Le plancher OWASP / STRIDE reste actif : seules la profondeur et la rigueur sont allégées.

Flux A2A & Humain. Chemin court ; revue allégée mais plancher sécurité non nul ; validation humaine conservée.



Jetable, allégé — plancher sécurité conservé

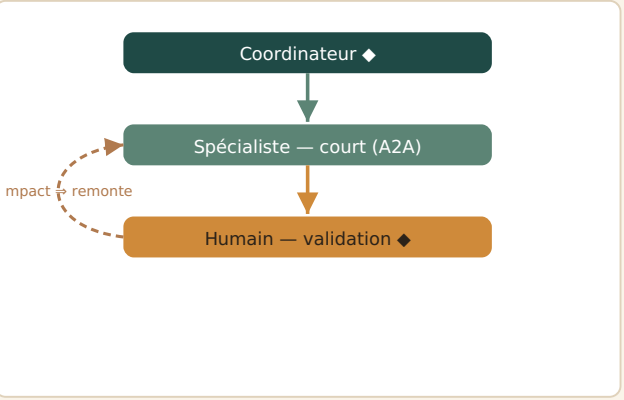
express

chemin court

Mots-clés : express, petit changement, rapide, mineur, trivial, correction rapide — **Depth minimal** **Vérif. standard**

Petit changement clair et à faible risque : chemin court, les stages lourds sont ignorés. **Garde-fou R1** : l'allègement est réservé aux changements sans impact runtime / production — dès qu'un express déploie ou effectue une action à impact, le découpage repasse à activé et la vérification remonte à standard minimum.

Flux A2A & Humain. Délégation directe, revue allégée (sans jamais contourner le plancher OWASP / STRIDE), validation humaine conservée.



Chemin court ; remonte à standard dès un impact

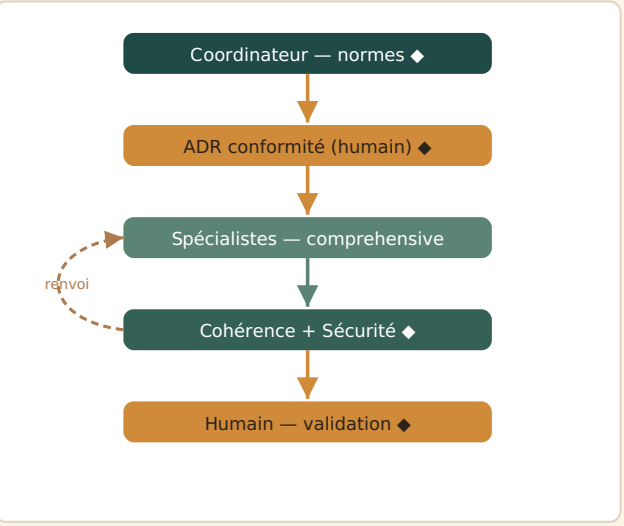
enterprise

conformité

Mots-clés : enterprise, conformité, PCI DSS, GDPR, RGPD, Loi 25, LPRPDE, structurant, réglementaire — **Depth comprehensive** **Vérif. renforcé**

Chantier structurant à fort impact ou à enjeu de conformité : parcours complet, Depth **comprehensive**, vérification renforcée. Point de contrôle obligatoire « **applicabilité des normes** » (PCI DSS / GDPR / Loi 25 / LPRPDE), décision tracée en ADR (y compris « aucune norme requise ») ; classification des données traitée. Garde-fou non abaissable : Depth ≥ standard, Vérification ≥ renforcé.

Flux A2A & Humain. Flux complet avec ADR de conformité, revues renforcées, et validation humaine à chaque jalon.



Complet + ADR conformité + revues renforcées